SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
ACADEMIC HOMEWORK & ASSESSMENT SUBMISSION

Student Name:	RAM CHARAN VANGA
Register No:	RA2511027010164
Course Code & Title:	21CSC202J - OPERATING SYSTEMS
Task & Channel:	Banker's Algorithm & Deadlock Avoidance Implementation [GCR]

Assignment Description:

Implement the Banker's Algorithm for deadlock avoidance in C. Input matrices: Allocation, Max, Available. Provide safety sequence.

Technical Solution:

Solution: Banker's Algorithm Implementation
Safe Sequence verified: `P1 -> P3 -> P4 -> P0 -> P2`. Full C implementation compiled.

Submitted by: RAM CHARAN VANGA (RA2511027010164)
Department of Computer Science & Engineering (Big Data Analytics)

Verified Source Code Deliverables:

File: bankers_algorithm.c

```
// Banker's Safety Algorithm in C
// Student: RAM CHARAN VANGA (RA2511027010164)
#include <stdio.h>
#include <stdbool.h>

#define P 5 // Number of processes
#define R 3 // Number of resource types

void calculateNeed(int need[P][R], int maxm[P][R], int allot[P][R]) {
    for (int i = 0 ; i < P ; i++)
        for (int j = 0 ; j < R ; j++)
            need[i][j] = maxm[i][j] - allot[i][j];
}

bool isSafe(int processes[], int avail[], int maxm[][R], int allot[][R]) {
    int need[P][R];
    calculateNeed(need, maxm, allot);

    bool finish[P] = {0};
    int safeSeq[P];
    int work[R];
    for (int i = 0; i < R ; i++) work[i] = avail[i];

    int count = 0;
    while (count < P) {
        bool found = false;
        for (int p = 0; p < P; p++) {
            if (finish[p] == 0) {
// ... [Full code attached in repository]
```